Computer Science and Engineering

Networked Software for Distributed Systems

A Distributed, Event-Sourced Smart Grid Platform

Authors
Vivek Kumar Sharma
Shaurya Aditiya

Professor
Luca Mottola

Project
#1 — Distributed Smart Power Grid

a.y. 2025/26

1 Introduction

This document describes the design and implementation of **SmartGrid**, a distributed smart-power-grid management platform built as a set of independently deployable **Spring Boot** microservices coordinated exclusively through **Apache Kafka** (single-broker **KRaft** mode, no ZooKeeper), complemented by a **Spark Structured Streaming** job that provides real-time district-level analytics and an optional **C++/MPI** simulator able to inject synthetic telemetry at scale.

The guiding design principle is **event sourcing via choreography**: every service owns exactly one Kafka topic as its append-only write-ahead log, exposes a REST API backed purely by an in-memory projection of that log, and rebuilds this projection from offset zero on startup before serving a single request. No service ever calls another service synchronously over REST to read or validate its data; cross-service knowledge — e.g. `node-manager` confirming that a `userId` exists before accepting a new node — is obtained exclusively by subscribing to the owning service's topic and maintaining a private, read-only materialized view of it. This keeps every service independently deployable, restart-safe, and free of synchronous inter-service coupling, at the cost of eventual (rather than immediate) cross-service consistency.

Design Goals

Table 1 summarizes the platform's four main design goals and the mechanism that realizes each of them.

Goal	Guarantee	Mechanism
Durability	Any service can be killed and restarted with zero data loss	`@EventListener(ApplicationReadyEvent)` replays the service's own topic from offset 0 into memory before the REST layer accepts traffic
Decoupling	Services never block on one another; a slow or down peer cannot cascade	Cross-service reads are served by a private, live-updated cache built from a consumed topic (e.g. Node Manager's cache of `user-events`), never a synchronous REST call
Real-time analytics	District-level average energy balance and bounded accumulator state of charge, continuously updated	`analytics-service` (Spark Structured Streaming) runs two parallel queries over `measurement-events`: a sliding-window average and a capacity-bounded running sum
Load & scale testing	Synthetic, high-volume telemetry, plus a like-for-like comparison of task-allocation strategies	A standalone C++/MPI simulator publishes directly to `measurement-events` under three selectable strategies, one of which (`node_partition`) uses a real `MPI_Allreduce` to combine cross-rank contributions

Table 1: Design goals and their implementation.

2 Architecture

Service Catalog

- **Account Service** (:8081): owns `User` records; publishes `user-events` (`UserRegistered/Updated/Deleted`, keyed by `userId`) on every mutation from its `/users/*` REST API.
- **Node Manager** (:8082): owns `Node` records (id, owning user, district, producer/consumer/accumulator type); publishes `node-events` keyed by `nodeId`. Consumes `user-events` purely to validate that a node's `userId` exists before accepting it — a read-only materialized view, not a synchronous call to Account Service.
- **Measurement Service** (:8083): stateless ingestion point for raw meter readings; a private cache replays `node-events` to enrich each reading with its district/type before publishing the canonical `measurement-events` (`MeasurementReported`, keyed by `districtId` for per-district ordering).
- **Billing Service** (:8084): accumulates per-user kWh usage in memory from `measurement-events` (resolving `nodeId`→`userId` through its own replayed `node-events` cache); a `@Scheduled` task drains the accumulator every 60 s and emits `billing-events` (`UsageRecordCreated`) at a fixed tariff.
- **Analytics Service**: not a Spring Boot service but a **Spark Structured Streaming** job (`spark-submit`). It signs each measurement (producer +, consumer −) and runs two continuous queries over `measurement-events`: a 10-minute/5-minute sliding-window *average* per district into `district-window-stats`, and a *complete*-mode running sum, per district, of accumulators' own capacity-clamped charge deltas into `district-soc-state` — bounded and physically meaningful, since each delta was already clamped to $[0, \text{capacity}]$ at the source (the C++ simulator, described below).
- **Presentation Service** (:8085): the system's pure CQRS *read side*. It produces nothing; it consumes

`node-events`, `billing-events` and `district-window-stats/district-soc-state`, stitching all three into the REST surface (`/users/{id}/nodes`, `/users/{id}/bills`, `/analytics/districts`) consumed by the dashboard.

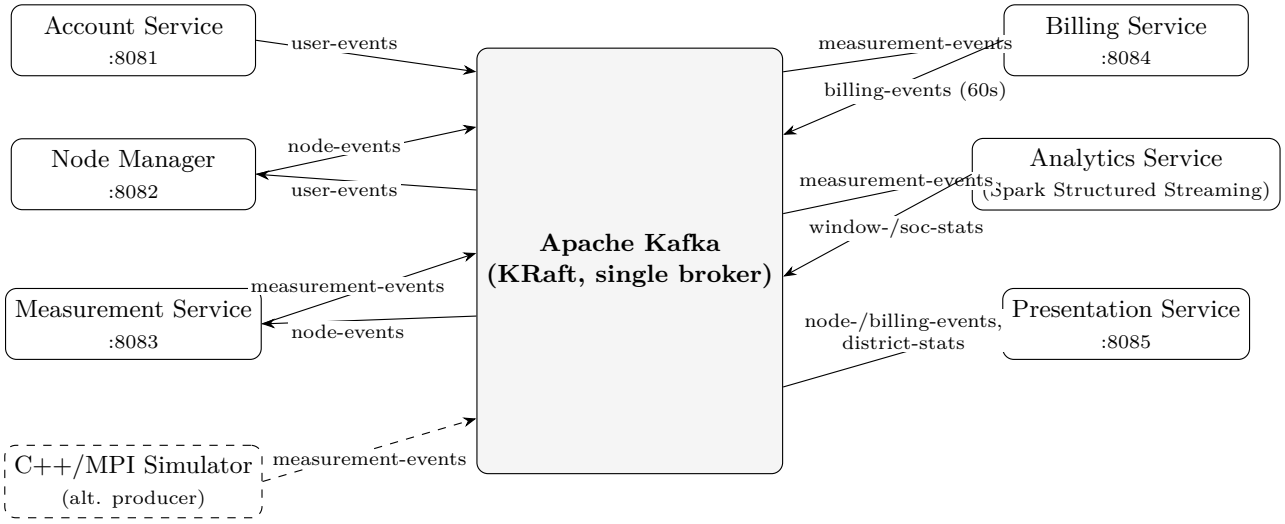

Figure 1: Publish/subscribe topology: every arrow is a Kafka topic, never a direct service call. Solid arrows are the REST-ingestion path; the dashed arrow shows the C++/MPI simulator publishing directly into `measurement-events` as an alternate, higher-volume producer. A vanilla-JS dashboard (not shown) additionally polls Account Service, Node Manager and Presentation Service directly over REST to render the UI.

State Ownership

No two services ever hold the same row of truth: each owns exactly the state derived from the topic(s) it consumes, and Kafka — not any service's database — is the durable system of record. This makes Presentation Service a textbook CQRS query side: three independently owned write models (`node-events`, `billing-events`, `district-*-stats`) are joined only in the read projection, never in the write path.

3 Event Sourcing, Choreography, and Fault Recovery

Replay-Then-Listen: One Pattern, Every Service

Every stateful component in the system — Account Service's `User` store, Node Manager's cache of `user-events`, Measurement Service's and Billing Service's caches of `node-events`, Billing Service's accumulator over `measurement-events`, and Presentation Service's three consumed views — is built with the same idiom: on `ApplicationReadyEvent`, a dedicated replay routine opens a disposable Kafka consumer with a fresh, unique `group.id` and `auto.offset.reset=earlies` drains the topic from the beginning into the in-memory store, and only then does the service register its live `@KafkaListener` and start accepting REST traffic. This is what makes every service **restart-safe with zero persistent storage**: a service's entire state is, by construction, nothing more than a cached fold over its topic(s).

A Subtlety in "Caught Up" Detection

The replay loop must decide *when* it has drained the topic's backlog, since Kafka never signals end-of-log explicitly. The natural heuristic — stop after N consecutive empty `poll()` calls — has a race condition: the very first polls of a fresh, uniquely-named consumer group return empty *because the group is still rebalancing and has not yet been assigned a partition*, not because the topic is empty. If those rebalance-time empty polls are counted toward the threshold, the loop can conclude "caught up" at the exact moment partitions are finally assigned — before a single historical record has been read — silently replaying zero events on an otherwise healthy restart. The fix is to gate the empty-poll counter on `consumer.assignment()` being non-empty, so only genuine post-assignment empty polls count toward completion. This is the kind of correctness detail that a unit test over a mocked consumer will not catch, but a real kill-and-restart integration test will — confirmed in practice: every replay routine in the system (eight in total, one per cache) hit this exact race on a live restart before the fix, each recovering zero records despite a healthy topic.

Publish Means Acknowledged, Not Attempted

Every producer blocks on the `Future` returned by `KafkaTemplate.send(...)` and logs the resulting topic-partition-offset, instead of firing-and-forgetting it: a fire-and-forget publish can return `201 Created` before

Kafka has durably accepted the write, silently diverging the caller's belief from the durable log until a later replay quietly comes up short. Controllers also publish *before* committing to their own in-memory store, so a publish failure never leaves the two disagreeing.

Choreography Over Orchestration

No component acts as a saga coordinator or orchestrator: cross-service consistency emerges purely from independent replay of shared topics. The clearest example is Node Manager validating a node's `userId`: rather than issuing a synchronous `GET` to Account Service, which would make node creation unavailable whenever Account Service happens to be down, it consults its own continuously-updated `user-events` projection. The trade-off is explicit: a `userId` that was just registered may not yet be visible to Node Manager if its consumer lags, so validation is eventually, rather than immediately, consistent — an acceptable cost given the system's read-heavy, analytics-oriented workload.

Time-Triggered Aggregation in Billing Service

Billing Service deliberately mixes two consistency models: it accumulates usage *continuously* from the live `@KafkaListener` on `measurement-events`, but only *emits* the resulting `billing-events` on a fixed 60-second `@Scheduled` cadence. This decouples the high-frequency, per-reading input rate from the low-frequency, per-user output rate, avoiding one `billing-events` message per raw measurement while still keeping the underlying usage total always up to date in memory.

4 Implementation and Verification

Technology Stack

- **Microservices:** Java 21, Spring Boot, one Maven module per service under a single reactor `pom.xml`; each exposes a small REST controller and one or more Kafka producer/consumer components, with no shared library beyond the event JSON shapes documented in `kafka-topics.md`.
- **Messaging:** Apache Kafka 7.6 in **KRaft** mode (self-managed metadata quorum, no ZooKeeper), a single broker/controller node for this deployment, six topics in total (`user-`, `node-`, `measurement-`, `billing-events`, `district-window-stats`, `district-soc-state`).
- **Analytics:** Apache Spark 3.5 Structured Streaming (Java 17), submitted as a plain `spark-submit` job rather than a Spring service, reading and writing Kafka directly via the `spark-sql-kafka-0-10` connector.
- **Load simulation:** a standalone C++17/CMake project (`simulation/`) modeling grid districts of producer/consumer/accumulator nodes (no-MPI stub as fallback); with `librdkafka` it publishes directly to `measurement-events` at higher volume than the REST path. Three runtime-selectable strategies satisfy the assignment's dense/sparse comparison: *round-robin* and *weighted* (load-balanced by node count) assign *whole districts* per rank — zero inter-rank communication. *Node-partition* splits *individual nodes* across ranks instead: each rank sums only its own nodes, then `MPI_Allreduce` combines all partial sums into the true district balance — real, measurable communication overhead versus the other two strategies' zero baseline. Results are tagged by (scenario, strategy, process count) for direct comparison.
- **Deployment:** Docker Compose orchestrates all seven containers (Kafka + five Spring services + the Spark job); a dependency-free vanilla-JS dashboard (`frontend/index.html`) talks directly to Account Service, Node Manager and Presentation Service over REST for the UI.

Verification

The system was verified end to end against the live Docker Compose stack, not only at the unit level:

- **Ingestion chain:** an automated script registers a user, adds a node for it, and submits a measurement, confirming at each step that the downstream service correctly consumed the upstream event — e.g. Node Manager accepting the node *because* it had already replayed the matching `UserRegistered` event.
- **Fault recovery:** all eight replay/cache routines across all five services were killed and restarted mid-test after real data had been produced, and each fully recovered its state from Kafka alone.
- **Streaming analytics:** the Spark job was confirmed to consume live `measurement-events` and produce correctly-shaped, bounded output on `district-soc-state`.
- **Distributed simulation correctness:** for *node-partition*, the `MPI_Allreduce`-combined balance matched an independent manual sum of every rank's recorded per-node deltas to floating-point precision.

Conclusion

By making Kafka the single, durable source of truth and requiring every service to derive its state purely by replaying topics it does not own, the system achieves independent deployability, zero-persistent-storage restart safety, and a clean separation between transactional services, streaming analytics, and load generation.